Simulation de dynamique moléculaire en C++

IKNE Mouad & **TOUATI Ayoub**

Résumé — Nous présentons l'implémentation en C++17 d'un simulateur de dynamique moléculaire bidimensionnel basé sur le schéma d'intégration de Störmer-Verlet et la méthode des cellules liées. Le simulateur est appliqué à l'étude de l'impact d'un disque circulaire de 395 particules sur une membrane de 17 227 particules, toutes en interaction via le potentiel de Lennard-Jones tronqué à $r_{\text{cut}} = 2.5\,\sigma$. L'adoption de la méthode des cellules liées ramène la complexité du calcul des forces de $O(N^2)$ à $O(N)$, rendant la simulation de systèmes à plusieurs dizaines de milliers de particules praticable. Une analyse de profil d'exécution via `gprof` révèle que 27.6% du temps est consommé par les accesseurs de position, suggérant des pistes d'optimisation à mémoire réduite.

Mots-clés : dynamique moléculaire, Lennard-Jones, cellules liées, Störmer-Verlet, C++17, CMake, GoogleTest, gprof, VTK, Paraview.

1. Introduction

La dynamique moléculaire (DM) est une méthode de simulation numérique qui consiste à intégrer les équations du mouvement newtonien pour un ensemble de N particules en interaction. Elle est au cœur de nombreux domaines : modélisation de matériaux, chimie computationnelle, biophysique. Le défi principal de la DM réside dans le passage à l'échelle : un calcul naïf des forces entre paires est en $O(N^2)$, ce qui devient prohibitif dès que N dépasse quelques milliers.

Ce projet s'inscrit dans le cours de C++ pour les mathématiques appliquées de l'ENSIMAG. L'objectif était de construire, de zéro, un simulateur de DM 2D complet : conception orientée objet, séparation interface/implémentation, système de build CMake, tests unitaires GoogleTest, documentation Doxygen, et analyse de performance par profiling. La physique visée est l'impact d'un disque circulaire de particules sur une membrane horizontale.

2. Modèle physique

Potentiel de Lennard-Jones

Le potentiel de Lennard-Jones (LJ) modélise l'interaction entre deux particules neutres séparées d'une distance r :

$$V_{\text{LJ}}(r) = 4\varepsilon\left[\left(\frac{\sigma}{r}\right)^{12} - \left(\frac{\sigma}{r}\right)^{6}\right] \tag{1}$$

Le terme en r^{-12} modélise la répulsion de Pauli ; le terme en r^{-6} capture l'attraction de van der Waals. Le minimum d'énergie se situe en $r^* = 2^{1/6}\,\sigma \approx 1.12\,\sigma$, avec $V(r^*) = -\varepsilon$.

La force correspondante, obtenue par $\mathbf{F} = -\nabla V$, est :

$$F(r) = \frac{24\varepsilon}{r}\left[2\left(\frac{\sigma}{r}\right)^{12} - \left(\frac{\sigma}{r}\right)^{6}\right] \tag{2}$$

On applique une troncature nette à $r_{\text{cut}} = 2.5\,\sigma$. À cette distance, $|V_{\text{LJ}}(r_{\text{cut}})| < 0.02\,\varepsilon$, ce qui est physiquement négligeable.

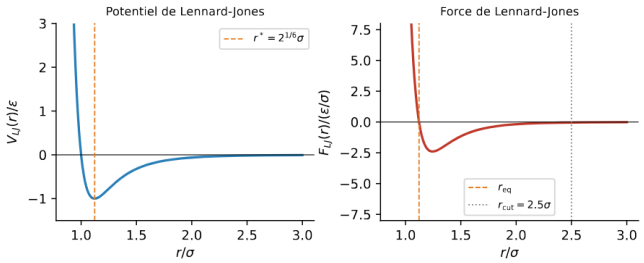

Figure 1: Potentiel (gauche) et force (droite) de Lennard-Jones en unités réduites. La ligne orange marque l'équilibre $r^* = 2^{1/6}\sigma$; la ligne pointillée grise indique le rayon de coupure $r_{\text{cut}} = 2.5\sigma$.

Potentiel gravitationnel et conditions aux limites

Un potentiel gravitationnel de constante $G = -12$ est appliqué selon l'axe y, simulant la chute libre du disque vers la membrane. Les conditions aux limites sont de type réflexion spéculaire sur les quatre parois du domaine $[0, L_x] \times [0, L_y]$.

Thermostat par rescaling

Pour éviter la divergence de l'énergie cinétique lors des impacts, on applique tous les 10^3 pas un rescaling des vitesses :

$$\mathbf{v}_i \leftarrow \mathbf{v}_i \cdot \sqrt{\frac{E_{c,\text{cible}}}{E_c}} \tag{3}$$

avec $E_{c,\text{cible}} = 0.005 \times (N_1 + N_2)$.

3. Méthodes numériques

Intégration de Störmer-Verlet

Le schéma de Störmer-Verlet (velocity-Verlet) est un intégrateur symplectique d'ordre 2 :

$$\mathbf{x}_i(t+\delta t) = \mathbf{x}_i(t) + \delta t\,\mathbf{v}_i(t) + \frac{\delta t^2}{2m_i}\,\mathbf{F}_i(t) \tag{4}$$

$$\mathbf{v}_i(t+\delta t) = \mathbf{v}_i(t) + \frac{\delta t}{2m_i}\left[\mathbf{F}_i(t) + \mathbf{F}_i(t+\delta t)\right] \tag{5}$$

Sa nature symplectique garantit la conservation de l'énergie sur le long terme, contrairement à Euler ex-

Algorithm 1: Boucle principale de Störmer-Verlet

Input: Univers $\mathcal{U}$, δt, t_{end}, $E_{c,\text{cible}}$
Initialisation : $t \leftarrow 0$, calcul des forces initiales;
while $t < t_{\text{end}}$ **do**
  Mise à jour des positions selon (4);
  Redistribution dans les cellules (`maj_cellules`);
  Recalcul des forces $\mathbf{F}(t + \delta t)$;
  Mise à jour des vitesses selon (5);
  if step mod 1000 = 0 **then**
    Rescaling des vitesses (3);
    Sauvegarde frame VTK;
  $t \leftarrow t + \delta t$;

plicite. Pour $\delta t = 5 \times 10^{-4}$ et $t_{\text{end}} = 29.5$, cela représente 59 000 itérations.

Méthode des cellules liées

La méthode des cellules liées réduit la complexité du calcul des forces de $O(N^2)$ à $O(N)$ en exploitant la troncature du potentiel. L'espace est découpé en cellules de côté r_{cut}. Pour chaque particule, seules les particules des cellules voisines (9 au maximum en 2D) sont candidates à l'interaction.

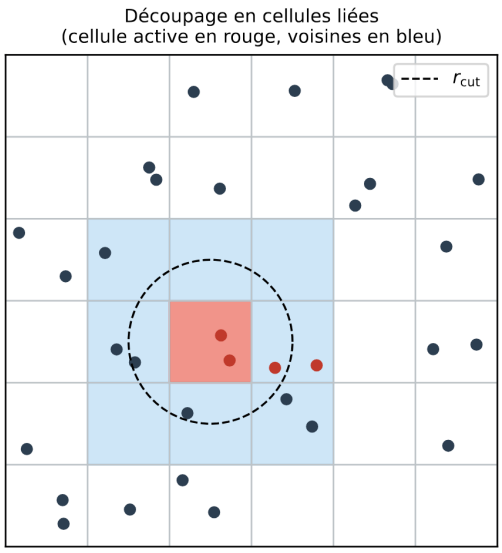

Figure 2: Principe des cellules liées. La cellule active (rouge) interagit uniquement avec ses 8 voisines (bleu). Le cercle en pointillé représente r_{cut}.

En supposant une densité uniforme, chaque cellule contient en moyenne $\bar{n} = N/(N_x N_y)$ particules. Le coût par pas est alors $O(9\,\bar{n} \cdot N) = O(N)$ si $N_x, N_y = O(\sqrt{N})$.

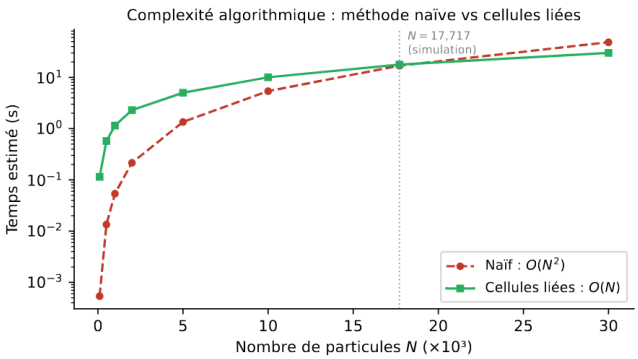

Figure 3: Comparaison des complexités algorithmiques. Le gain en faveur des cellules liées devient écrasant dès $N \gtrsim 5\,000$ particules.

4. Architecture logicielle

Organisation du projet

Répertoire	Contenu
`include/`	Déclarations (.hxx)
`src/`	Implémentations (.cxx)
`demo/`	Programme de démonstration
`test/`	Tests unitaires GoogleTest
`doc/`	Documentation Doxygen
`build/`	Artefacts compilés (ignoré Git)

Hiérarchie des classes

Le système est articulé autour de cinq abstractions :

- `Particule` : état physique (position, vitesse, force, masse, id, catégorie).
- `vecteur` : algèbre 3D (norme, produit scalaire, opérateurs arithmétiques).
- `Cellule` : conteneur `std::vector<Particule>` + liste d'indices de voisines.
- `Grille` : tableau 1D de cellules avec indexation row-major et initialisation des voisinages.
- `Univers` : orchestrateur – calcul des forces, intégration, conditions aux bords, énergie.

Choix techniques notables

Conteneurs : Un `std::vector<Particule>` par cellule offre un accès contigu en mémoire. La grille utilise un vecteur plat avec indexation `idx` $= x + N_x \cdot y$, évitant le surcoût d'un tableau 2D de vecteurs.

Sorties VTK : Les états du système sont exportés au format `UnstructuredGrid` VTK, directement lisibles par Paraview pour l'animation de la simulation.

Paramètres de la simulation

Paramètre	Valeur	Description
L_x	250	Largeur du domaine
L_y	180	Hauteur du domaine
ε	1.0	Énergie LJ
σ	1.0	Longueur caractéristique
r_{cut}	2.5	Rayon de coupure
δt	5×10^{-4}	Pas de temps
t_{end}	29.5	Temps final
G	-12	Constante gravitationnelle
N_1	395	Particules du disque
N_2	17 227	Particules de la membrane
m_{disque}	1.0	Masse (disque)
m_{membrane}	1.00	Masse (membrane)
$\mathbf{v}_0$	$(0, 10)$	Vitesse initiale du disque
$E_{c,\text{cible}}$	$0.005\,(N_1 + N_2)$	Énergie cinétique cible

5. Analyse de performance

Résultats du profiling gprof

La simulation complète a été exécutée en mode `-pg` sur une machine portable standard. Le temps total d'exécution est de **16.94 s** pour 59 000 pas de temps et $N = 17\,717$ particules. La figure 4 détaille la répartition du temps.

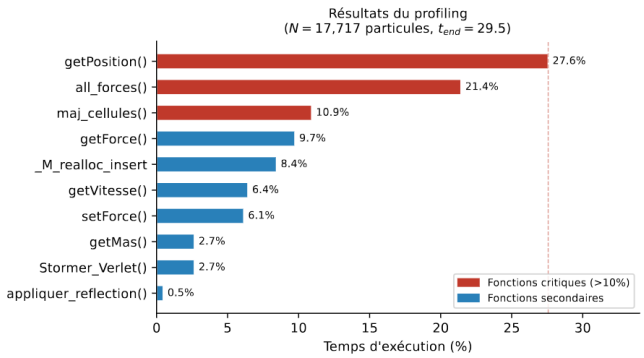

Figure 4: Profil d'exécution gprof — répartition du temps par fonction. Les trois fonctions critiques (rouge) concentrent **60%** du temps total.

Analyse des fonctions critiques

Fonction	% temps	Appels
`getPosition()`	27.6%	2.46×10^9
`all_forces()`	21.4%	1 901
`maj_cellules()`	10.9%	1 900
`getForce()`	9.7%	7.87×10^8
`_M_realloc`	8.4%	63 032

Le résultat le plus frappant est que `getPosition()` – un simple accesseur – concentre à lui seul 27.6% du temps, avec 2.46×10^9 appels. Ce n'est pas le corps de la fonction qui est coûteux, mais le volume d'appels : chaque interaction LJ nécessite quatre appels (deux particules × deux coordonnées). Inliner ces accesseurs via `inline` ou `[[gnu::always_inline]]` pourrait éliminer ce surcoût.

La fonction `_M_realloc_insert` (8.4%) révèle des réallocations fréquentes lors de `maj_cellules`. Pré-réserver les vecteurs de cellules avec `reserve()` éliminerait ces copies.

Pistes d'optimisation

1. Inlining des accesseurs. Déplacer `getPosition()`, `getForce()`, `setForce()` dans les headers avec `inline`. Gain estimé : ∼30%.
2. Pré-réservation des vecteurs.`cell.reserve(max)` évite les réallocations dans `maj_cellules`. Gain : ∼8%.
3. Parallélisation OpenMP. La boucle externe de `all_forces` est parallélisable sur les cellules non-adjacentes.
4. **Éviter `std::pow`.** Remplacer `pow(r2,6)` par des multiplications successives ($r^2 \to r^4 \to r^6$).
5. **Structure SoA.** Passer d'un *Array of Structures* à un *Structure of Arrays* améliore la localité de cache.

6. Tests et validation

Stratégie de test

Les tests unitaires sont écrits avec GoogleTest, récupéré automatiquement par CMake via `FetchContent`. Ils couvrent les cinq classes principales :

- `Particule` : constructeurs, getters/setters, force d'interaction, cas limite de singularité ($r \to 0$).
- `vecteur` : opérations algébriques, norme, produit scalaire.
- `Cellule` : ajout/suppression, gestion des voisins.
- `Grille` : indexation, initialisation des voisinages (4/6/9 voisins selon la position).
- `Univers` : ajout de particules, réflexion aux bords, énergie cinétique, rescaling.

Cas limites critiques

Trois cas piégeux ont nécessité une attention particulière.

Division par zéro dans le calcul LJ lorsque $r < r_{\min}$: la force est saturée à ± 5000, évitant l'explosion numérique.

Suppression en cours d'itération dans `appliquer_absorption` : l'usage du retour de `erase()` comme nouvel itérateur est indispensable pour éviter un comportement indéfini.

Particules en bordure exacte du domaine : une particule à la frontière exacte ne doit pas être affectée à une cellule hors-grille.

Un **détecteur de NaN** est intégré dans la boucle principale pour arrêter proprement la simulation en cas de divergence numérique.

7. Difficultés et retour d'expérience

Sur le plan numérique, la calibration de δt a demandé plusieurs itérations. Un δt trop grand rend Verlet instable (énergie divergente) ; un δt trop petit multiplie le coût de calcul. L'erreur de signe dans le gradient du potentiel LJ (attraction/répulsion inversées) a causé des

simulations qui explosaient immédiatement. Un test unitaire dédié a permis de l'identifier rapidement.

Plusieurs axes d'amélioration ont été identifiés en cours de projet, inlining des accesseurs, pré-réservation des vecteurs, parallélisation OpenMP, et nous aurions souhaité les implémenter pour livrer une version encore plus aboutie. La période de rendu coïncidant avec les examens, nous n'avons pas pu y consacrer le temps supplémentaire qu'ils auraient mérité. Les priorités ont dû être arbitrées, et nous avons préféré consolider ce qui existait plutôt que d'introduire de nouvelles fonctionnalités au risque de déstabiliser un code qui fonctionne.

8. Conclusion

Nous avons développé un simulateur de dynamique moléculaire 2D complet en C++17, capable de simuler l'impact d'un disque de 395 particules sur une membrane de 17 227 particules en moins de 17 secondes sur un PC portable. L'adoption de la méthode des cellules liées est la contribution algorithmique centrale, réduisant la complexité de $O(N^2)$ à $O(N)$.

L'analyse de profil gprof a révélé un goulot d'étranglement inattendu : les accesseurs de position concentrent 27.6% du temps d'exécution en raison du volume d'appels (2.46×10^9). L'inlining de ces accesseurs constitue l'optimisation au plus fort potentiel de gain.

Au-delà du résultat, ce projet a été l'occasion de maîtriser un écosystème de développement C++ complet (CMake, GoogleTest, Doxygen, gprof, Git, Paraview), une expérience dont la valeur dépasse celle du simulateur lui-même.